Entendendo o exemplo do zero

1. Qual é o problema?

Queremos somar 1 + 2 + 3 + ... + N. Se N=10, esperamos 55. Dividimos os números entre membros do Hazelcast, cada um calcula uma parte e o cliente junta as respostas.

Isso é uma **redução por soma**: transformar vários valores em um total. Não é all-reduce, porque o resultado final fica no cliente e não é automaticamente enviado a todos os membros. A aplicabilidade é didática: mostra a base de agregações de vendas, contadores e estatísticas, embora somar essa sequência específica não precise de distribuição na prática.

2. Quais conceitos do Hazelcast preciso conhecer?

Conceito	Ideia simples	Onde aparece
Membro	Processo que participa do cluster e guarda uma parte dos dados	Main.membro
Cliente	Programa que manda dados e pede operações ao cluster	Cliente
IMap	Mapa chave–valor distribuído entre os membros	getMap("numeros")
Callable	Uma tarefa que executa e devolve uma resposta	SomaTask
IExecutorService	Serviço que manda a tarefa executar no cluster	submitToAllMembers
Backup	Cópia dos dados em outro membro	setBackupCount(1)

3. Guardar os números

No cliente:

```
IMap<Integer, Long> mapa = hz.getMap("numeros");
Map<Integer, Long> entrada = new HashMap<>();
for (int i = 1; i <= n; i++) entrada.put(i, (long) i);
mapa.putAll(entrada);
```

entrada é um mapa comum dentro do cliente. mapa é o mapa Hazelcast. Ao chamar putAll, enviamos os pares ao cluster. A chave 1 guarda o valor 1, a chave 2 guarda o valor 2 e assim por diante. Quem decide o membro que fica com cada chave é o Hazelcast.

O método completo também apaga a entrada anterior. Por isso, não execute carregar enquanto outra pessoa estiver somando. O laboratório usa um conjunto de dados por vez.

4. Somar a parte de um membro

Este é o núcleo de SomaTask.call():

```
IMap<Integer, Long> numeros = hazelcast.getMap("numeros");
long soma = 0;
for (Integer chave : numeros.localKeySet()) {
    soma += numeros.get(chave);
}
return soma;
```

localKeySet() retorna as chaves primárias pertencentes ao membro que está executando a tarefa. Não retorna também as cópias de backup para somá-las de novo. O código entregue verifica se um valor ficou indisponível e sinaliza erro para não produzir uma resposta silenciosamente incompleta.

A classe implementa três interfaces: Callable<Long> indica uma tarefa que retorna um número; Serializable permite enviar a tarefa; HazelcastInstanceAware permite ao Hazelcast fornecer à tarefa a instância local. Não precisamos criar threads manualmente.

5. Pedir a soma a todos os membros

```
var respostas = hz.getExecutorService("soma")
```

```
                .submitToAllMembers(new SomaTask());
long total = 0;
for (var resposta : respostas.values()) {
    total += resposta.get();
}
```

Cada resposta é um Future: uma resposta que pode ainda estar sendo calculada. get() espera o resultado. No código completo, há um prazo de 30 segundos por resposta para a demonstração não ficar esperando indefinidamente. O cliente também mostra as somas parciais por endereço.

6. Exemplo para apresentar

Imagine, apenas para explicar, que um membro recebeu 1, 4 e 7; outro recebeu 2, 5 e 8; e outro recebeu 3, 6, 9 e 10. As somas seriam 12, 15 e 28, e o cliente obteria 55. **A distribuição real usa hash e pode ser diferente**; não programamos essa sequência de distribuição.

7. Como conferir?

Para os valores de 1 até N, a resposta conhecida é N*(N+1)/2. A fórmula é utilizada somente para **verificar** o resultado. O cálculo distribuído percorre de fato o mapa. Usamos long: 100 mil números já somam mais do que cabe em um int.

A validação é para esta entrada conhecida e imutável. Igualdade da soma não prova integridade geral de um conjunto arbitrário, porque valores diferentes podem produzir o mesmo total. O trabalho não pretende ser um verificador universal de integridade.

8. O que muda se um membro cair?

O mapa tem um backup em outro membro. Com cópias sincronizadas, o Hazelcast recupera os dados no cluster. Mas uma tarefa de soma em andamento pode falhar. No modo acompanhar, o cliente permanece aberto e a próxima rodada faz a soma inteira novamente. Essa abordagem é mais fácil de entender que recuperar pequenos pedaços de trabalho.

A rodada pode ser rejeitada se a composição do cluster mudou ou se a soma divergir. estado/pronto ajudam a esperar a migração terminar. Não há uma fotografia transacional do mapa: durante mudanças, a iteração pelas chaves locais não representa necessariamente um conjunto estável. Não mude os valores durante a soma.

9. O que foi adaptado do site?

Fonte: https://docs.hazelcast.com/hazelcast/5.6/computing/executor-service

O exemplo oficial usa SumTask, um mapa, localKeySet() e uma soma local. Mantivemos essa estrutura. As adaptações são nomes em português, chaves inteiras, valores long, cliente que combina parciais, entrada 1..N com fórmula de conferência e comandos para o workshop. A configuração TCP, inspeção, acompanhamento e benchmark são apoio ao enunciado; não são conceitos adicionais do algoritmo.

10. O que estudar primeiro?

Primeiro explique N=10 e as três etapas: guardar, somar localmente, juntar. Depois leia SomaTask.java inteira. Em seguida, leia apenas carregar e somar em Cliente.java. Deixe os métodos de benchmark, inspeção e os scripts para a etapa de reprodução dos experimentos.